

# FixIt Hostel – Maintenance Complaint Triage

Design and Implementation of a Multi-Step Prompt Workflow

|                           |                                        |
|---------------------------|----------------------------------------|
| <b>Name</b>               | Yeluru Manoj Kumar                     |
| <b>Student ID</b>         | 99230040185                            |
| <b>Programme</b>          | Gen AI                                 |
| <b>Module</b>             | Prompt Engineering                     |
| <b>Session</b>            | Session 7 – Multi-Step Prompt Workflow |
| <b>Date of Submission</b> | 05-10-2026                             |

# Table of Contents

- 1. Introduction** 3
- 2. Objective** 3
- 3. Workflow Design** 3
- 4. Methodology** 4
  - 4.1 Step 1: Issue Entity Extraction Using Few-Shot Prompting .....4
  - 4.2 Step 2: Priority-Tier Assessment Using Chain-of-Thought Reasoning.....5
  - 4.3 Step 3: Structured Output Using JSON .....7
- 5. Results and Discussion** 8
- 6. Conclusion** 9

# 1. Introduction

This report presents the design and implementation of a three-step prompt workflow for FixIt Hostel, a maintenance request app used by residents of a college hostel. Residents currently write complaints in free text, and the wardens read them one by one. The workflow takes an unstructured complaint and progressively transforms it into a structured ticket that the app's dispatch layer can act on.

The workflow follows this sequence:

*Raw Complaint -> Issue Entity Extraction -> Priority-Tier Assessment -> Structured JSON Output*

Each stage applies a prompting technique suited to its purpose:

- Few-shot prompting to extract the details that matter from the complaint.
- Chain-of-thought reasoning to decide how urgently the maintenance team should respond.
- JSON formatting to produce a structured, machine-readable output that the dispatch backend can consume.

# 2. Objective

- Extract the operationally relevant information from an unstructured complaint using few-shot prompting.
- Analyse the extracted information and determine the appropriate response tier using structured reasoning.
- Convert the results into a valid JSON object that FixIt Hostel's dispatch layer could process.
- Demonstrate that the output of each stage is used as the input for the next.

# 3. Workflow Design

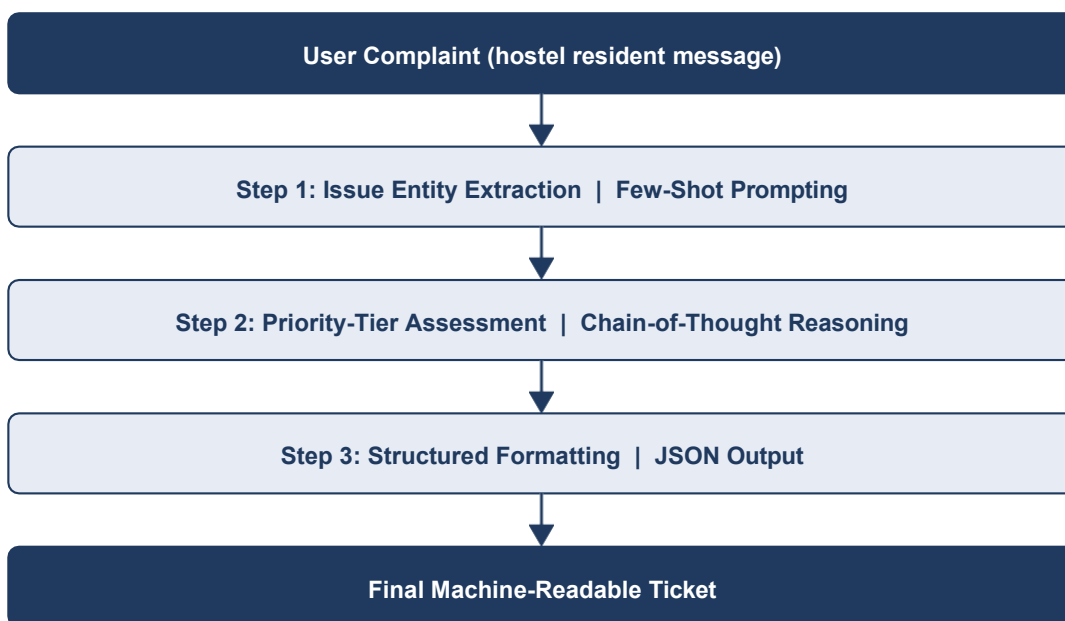


Figure 1: Prompt Chaining Workflow for FixIt Hostel's Complaint Triage

Separating the workflow into three stages ensures that each prompt has a single, focused responsibility. Step 1 identifies what is actually wrong and how risky it is; Step 2 makes a judgement about how quickly the issue must be handled; and Step 3 packages the result for the dispatch backend.

This separation keeps each individual prompt simple and makes the workflow easier to inspect. This is important for a safety-related service, since every intermediate judgement can be reviewed on its own rather than trusting one opaque response.

## 4. Methodology

The workflow was implemented using Google AI Studio. The following resident complaint was used as the input for the experiment:

*“The ceiling fan in my room 214 has been making a loud grinding noise since last night, and today it sparked when I switched it on. My roommate says it's fine, but I'm scared to use it. I haven't told the warden yet because the last complaint I made took two weeks to get fixed.”*

### 4.1 Step 1: Issue Entity Extraction Using Few-Shot Prompting

#### Purpose

The purpose of the first stage was to extract the most important information from the resident's unstructured complaint. Few-shot prompting was chosen because worked examples demonstrate the relationship between an input and its desired output, while establishing a consistent format for the extracted information. Three examples spanning a minor issue, a moderately disruptive issue, and a high-hazard issue were provided before the actual complaint.

#### Prompt Used

I'm building the entity-extraction stage of a complaint triage workflow for a hostel maintenance app called FixIt Hostel. For this step, I need you to read a resident's complaint and pull out the details that matter for deciding how urgently the maintenance team should respond.

Use the examples below as a guide for what to look for and how to format your answer.

##### Example 1:

Resident complaint: “The tap in the common washroom on the third floor drips slowly. It has been like this for a few days. Not a big deal, but it wastes water.”

##### Output:

- Issue category: plumbing
- Location: third-floor common washroom
- Hazard indicators: none
- Duration: a few days
- Impact on residents: low (water wastage)
- Previous reports: none mentioned
- Resident tone: casual, mildly bothered

##### Example 2:

Resident complaint: “The Wi-Fi in Block B keeps dropping every evening for the past week. I can't attend my online classes properly and many others are facing the same problem.”

**Output:**

- Issue category: network
- Location: Block B
- Hazard indicators: none
- Duration: one week, recurring daily
- Impact on residents: moderate to high (multiple residents, academics)
- Previous reports: none mentioned
- Resident tone: frustrated

**Example 3:**

Resident complaint: "There is a strong smell of gas near the mess kitchen and a few of us are feeling dizzy."

**Output:**

- Issue category: gas leak
- Location: mess kitchen
- Hazard indicators: suspected gas leak, dizziness reported
- Duration: ongoing right now
- Impact on residents: critical (health symptoms already present)
- Previous reports: none mentioned
- Resident tone: alarmed

Now analyse this complaint using the same style: "The ceiling fan in my room 214 has been making a loud grinding noise since last night, and today it sparked when I switched it on. My roommate says it's fine, but I'm scared to use it. I haven't told the warden yet because the last complaint I made took two weeks to get fixed."

**Output Generated**

- Issue category: electrical (ceiling fan)
- Location: room 214
- Hazard indicators: grinding noise, sparking on switch-on (possible electrical fault)
- Duration: since last night, sparking observed today
- Impact on residents: moderate to high (fan unusable, shock or fire risk if used)
- Previous reports: none for this issue; past complaint took two weeks to resolve
- Resident tone: worried, hesitant to escalate

## 4.2 Step 2: Priority-Tier Assessment Using Chain-of-Thought Reasoning

**Purpose**

The purpose of the second stage was to determine how urgently the maintenance team should respond. Unlike entity extraction, judging urgency requires weighing several factors together, so the workflow used a structured reasoning approach that considers hazard to life or property, whether the hazard can be contained, how much daily life is disrupted, and whether the resident has lost trust in earlier reporting.

**Prompt Used**

Now that the important details have been extracted, the next step is to decide how urgently the FixIt Hostel maintenance team should respond to this complaint. Look at the information below and reason through the decision carefully. Consider whether there is any hazard to life or property, whether the hazard is active or can be avoided by the resident for now, how much the problem disrupts daily living, and whether earlier delays may have reduced the resident's trust in the reporting process.

Use the following guide:

- **Routine Maintenance:** the issue is minor or cosmetic, with no hazard; it can be scheduled in the normal weekly maintenance round.
- **Priority Repair:** the issue disrupts daily living for one or more residents but poses no safety hazard; it should be resolved within two to three days.
- **Urgent Safety Repair:** a safety hazard exists but can be contained by the resident (for example, by not using the device); a technician must attend the same day.
- **Emergency Escalation:** there is immediate danger to life, such as a gas leak, active fire, or live exposed wiring; the app must alert the warden and emergency services at once.

Explain your decision in a few numbered points. Base your answer only on the information provided below.

**Extracted complaint details:**

- Issue category: electrical (ceiling fan)
- Location: room 214
- Hazard indicators: grinding noise, sparking on switch-on (possible electrical fault)
- Duration: since last night, sparking observed today
- Impact on residents: moderate to high (fan unusable, shock or fire risk if used)
- Previous reports: none for this issue; past complaint took two weeks to resolve
- Resident tone: worried, hesitant to escalate

## Output Generated

1. The complaint describes a possible electrical fault: the grinding noise suggests mechanical failure, and the sparking on switch-on points to a short or damaged wiring. This is a real fire and shock risk, not just an inconvenience.
2. The hazard is not yet an immediate danger to life. The resident has stopped using the fan, and the fault can be contained by switching it off at the board, so this does not meet the criteria for Emergency Escalation.
3. Because a safety hazard is present, a Priority Repair timeline of two to three days would be too slow. The fault should be inspected the same day.
4. The resident's note that an earlier complaint took two weeks suggests they may stop reporting issues. A slow or generic acknowledgement would reinforce that, so the response should include a clear timeline and a follow-up check.

**Response tier: Urgent Safety Repair**

## Discussion

The complaint was classified as Urgent Safety Repair because sparking indicates a genuine electrical hazard, the hazard could be contained by the resident, and the resident's earlier experience of slow repairs meant that a vague reply would land poorly. This reasoning stage

worked from the information extracted in Step 1 rather than re-reading the original complaint independently, demonstrating the chaining relationship between the two prompts. The final tier decision was then passed to the third stage.

### 4.3 Step 3: Structured Output Using JSON

#### Purpose

The purpose of the final stage was to convert the results of the previous two stages into a structured format. JSON was chosen because it is consistently structured and easy for software systems to process. A result such as this is what FixIt Hostel's dispatch layer would consume to create a ticket, assign a technician, and notify the warden.

#### Prompt Used

Convert the results from the previous two stages of this FixIt Hostel complaint triage workflow into a valid JSON object. Return only the JSON object. Do not include explanations, headings, or markdown. Use exactly these three keys:

- "entities": a list of the important details extracted from the complaint
- "response\_tier": the final response tier decided during the previous step
- "recommended\_action": one to two sentences describing what the app and maintenance team should do, and why that fits the tier

#### Results from Step 1:

- Issue category: electrical (ceiling fan)
- Location: room 214
- Hazard indicators: grinding noise, sparking on switch-on (possible electrical fault)
- Duration: since last night, sparking observed today
- Impact on residents: moderate to high (fan unusable, shock or fire risk if used)
- Previous reports: none for this issue; past complaint took two weeks to resolve
- Resident tone: worried, hesitant to escalate

**Result from Step 2:** Response tier: Urgent Safety Repair

## Final JSON Output Generated

```
{
  "entities": [
    "Issue category: electrical (ceiling fan)",
    "Location: room 214",
    "Hazard indicators: grinding noise, sparking on switch-on (possible electrical fault)",
    "Duration: since last night, sparking observed today",
    "Impact on residents: moderate to high (fan unusable, shock or fire risk if used)",
    "Previous reports: none for this issue; past complaint took two weeks to resolve",
    "Resident tone: worried, hesitant to escalate"
  ],
  "response_tier": "urgent_safety_repair",
  "recommended_action": "FixIt Hostel should tell the resident to keep the fan switched off at the board and assign an electrician for a same-day inspection, because sparking indicates a fire and shock risk. The app should send a ticket with a clear arrival window and escalate to the warden if the ticket is not acknowledged within a few hours."
}
```

## 5. Results and Discussion

The three-stage workflow successfully processed the original complaint, carrying it from an unstructured message through to a structured JSON result that FixIt Hostel's backend could act on directly.

Table 1: Summary of the Prompt Workflow

| Step   | Task                           | Prompting Technique        | Output                       |
|--------|--------------------------------|----------------------------|------------------------------|
| Step 1 | Extract relevant issue details | Few-shot prompting         | Structured issue-detail list |
| Step 2 | Determine response tier        | Chain-of-thought reasoning | Response-tier classification |
| Step 3 | Format the results             | JSON output                | Machine-readable JSON object |

This separation makes the workflow easier to review, since each intermediate result can be inspected on its own. That matters for a hostel safety service, where it should always be possible to see why a complaint was ranked at a particular urgency. It also means the final output follows a consistent schema containing the three required fields:

- `entities`
- `response_tier`
- `recommended_action`

The resulting JSON object is well-formed and can be parsed by a standard JSON parser without requiring manual cleanup. Because the tier classification is grounded in explicit, inspectable reasoning (Step 2), the workflow could also be extended so that an Emergency Escalation tier automatically alerts the warden and emergency services rather than waiting in a queue.

## 6. Conclusion

This approach offers greater transparency, since intermediate outputs can be reviewed individually, and illustrates how large language models could be integrated into a hostel maintenance product like FixIt Hostel to make complaint handling faster, fairer, and safer.

- Few-shot prompting was used to extract relevant information from an unstructured resident complaint.
- Chain-of-thought reasoning was used to weigh hazard, containment, disruption, and prior reporting experience to determine the appropriate response tier.
- JSON formatting was used to convert the final results into a structured, machine-readable output.